Pattern Matching

Concepts of Programming Languages

CAS CS 320

Outline

- Discuss **pattern matching**
- This means giving a syntax, typing, and semantics
- This also means defining **patterns**, and describing what matching actually means

Pattern Matching

Pattern Matching (High Level)

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

Values come in different "shapes"

We want to do things based on the different shapes

Matching allows us to condition on the shape of the value

```
sum (1 :: 2 :: [])
```

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

failed match

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

successful match

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

create new bindings

```
1 + sum (2 :: [])
```

compute branch with new bindings

```
1 +
match 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

rinse and repeat

```
1 +
match 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

failed match

```
1 +
match 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

successful match

```
1 +
match 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

create new bindings

```
1 + (2 + sum [])
```

compute branch with new bindings

```
1 + (2 +
match [] with
| [] -> 0
| x :: xs -> x + sum xs)
```

rinse and repeat

```
1 + (2 +
match [] with
| [] -> 0
| x :: xs -> x + sum xs)
```

successful match

```
1 + (2 +
match [] with
| [] -> 0
| x :: xs -> x + sum xs)
```

no new bindings

1 + (2 + 0)

compute branch

3

fin

Making Things Formal

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

What do match-expressions look like? **(syntax)**

What is a valid match-expression? **(typing)**

How do match-expressions behave? **(semantics)**

Syntax

Match Syntax (Attempt)

```
<expr> ::= match <expr> with <arms>
<arms> ::= <arm> <arms> | <arm>
 <arm> ::= | <expr> -> <expr>
```

Match Syntax (Example)

```
match l with
| [] -> 0
| x :: xs -> x + sum xs
```

```
<expr> ::= match <expr> with <arms>
<arms> ::= <arm> <arms> | <arm>
 <arm> ::= | <expr> -> <expr>
```

Match Syntax (Example)

```
match 2 + 4 with
| 2 + 4 -> 2 * 4
| _ -> 0
```

Can we do this in OCaml?

Match Syntax

```
<expr> ::= match <expr> with <arms>
<arms> ::= <arm> <arms> | <arm>
 <arm> ::= | <pat> -> <expr>
 <pat> ::= ????????
```

Patterns (High Level)

```
match x with
| 1 -> e1
| 2 -> e2
| _ -> e3
```

A pattern describes the "shape" of a **value**

But we don't *just* care about the "shape" we also want to do stuff with the pieces of the data

<u>Patterns</u> are syntactic representations of values with names

Shallow vs. Deep Pattern Matching

```
let third l =
  match l with
  | _ :: _ :: x :: _ -> Some x
  | _ -> None
```

We're interested implementing *deep* pattern matching, which allows for *nested* patterns

We can also implement *shallow* pattern matching ADTs (which are sometimes called *eliminators*)

Shallow Pattern Matching for Lists

```
<expr> ::= match <expr> with
           | [] -> <expr>
           | <var> :: <var> -> <expr>
```

We can hardcode the possible patterns

This works, but is tedious

Shallow Pattern Matching for Lists

```
let third l =
  match l with
  | [] -> None
  | _ :: l -> (
      match l with
      | [] -> None
      | _ :: l -> (
          match l with
          | [] -> None
          | x :: _ -> Some x
        )
    )
```

Aside: Wildcards

```
let third l =
  match l with
  | _ :: _ :: x :: _ -> Some x
  | _ -> None
```

OCaml treats wildcards as distinct from variables

We'll treat "_" as a variable name (though we'll still have to treat them specially)

Pattern Syntax

```
 <pat> ::= () | true | false
         | [] | <pat> :: <pat>
         | <pat> , <pats>
         | <int> | <var> | ( <pat> )
<pats> ::= <pat> , <pats> | <pat>
```

Pattern Syntax (Example)

```
<pat> ::= () | true | false
        | [] | <pat> :: <pat>
        | <pat> , <pats>
        | <int> | <var> | ( <pat> )
<pats> ::= <pat> , <pats> | <pat>
```

Match Syntax

```
<expr> ::= match <expr> with <arms>
<arms> ::= <arm> <arms> | <arm>
 <arm> ::= | <pat> -> <expr>
 <pat> ::= () | true | false
         | [] | <pat> :: <pat>
         | <pat> , <pats>
         | <int> | <var> | ( <pat> )
<pats> ::= <pat> , <pats> | <pat>
```

Typing

Questions

What makes a valid match expression?

When should I get a type error from a pattern match?

Match Typing (Attempt)

Pattern Typing (High Level)

```
match 1 :: 2 :: 3 with
| [] -> 0
| true -> 2
| x :: _ -> x
```

type error

Pattern Typing (High Level)

```
match 1 :: 2 :: 3 with
| [] -> 0
| true -> 2
| x :: _ -> x
```

type error

Patterns are *typed*. We can check for "reasonableness" of patterns

Pattern Typing (High Level)

```
match 1 :: 2 :: 3 with
| [] -> 0
| true -> 2
| x :: _ -> x
```

type error

Patterns are *typed*. We can check for "reasonableness" of patterns

(In fact, we'll talk about check that we have exactly the right amount of patterns, this is *exhaustiveness checking*)

Pattern Typing (High Level)

```
match 1 :: 2 :: 3 with
| [] -> 0
| true -> 2
| x :: _ -> x
```

type error

Patterns are *typed*. We can check for "reasonableness" of patterns

(In fact, we'll talk about check that we have exactly the right amount of patterns, this is *exhaustiveness checking*)

Reminder: patterns can also *generate* variable declarations

$\{$`sum : int list -> int`$\}$ $\vdash$

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

$: \mathtt{int}$

$\{$`sum : int list -> int`$\}$ $\vdash$

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

$: \texttt{int}$

check both branches are ints

$\{$`sum : int list -> int`$\}$ $\vdash$

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

$: \texttt{int}$

check both branches are ints

$\{$`sum : int list -> int`$\}$ $\vdash$ `0` `: int`

$\{$`sum : int list -> int`$\}$ $\vdash$

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

$: \mathtt{int}$

$\{$`sum : int list -> int`$\}$ $\vdash$ `x + sum x` : `int`

$\{$`sum : int list -> int`$\}$ $\not\vdash$ `x + sum x` `: int`

We don't know what x and xs are

$\{$`sum : int list -> int`$\}$ $\vdash$

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

$: \texttt{int}$

$\{$`sum : int list -> int`$\}$ $\vdash$

```
match 1 :: 2 :: [] with
| [] -> 0
| x :: xs -> x + sum xs
```

$: \mathrm{int}$

the pattern generates new declarations

$$\left\{\begin{array}{l}\texttt{sum : int list -> int}\\ \texttt{\quad x : int}\\ \texttt{\quad x : int list}\end{array}\right\} \vdash \texttt{x + sum xs} : \texttt{int}$$

$$\left\{ \begin{array}{l} \texttt{sum : int list -> int} \\ \quad \texttt{x : int} \\ \quad \texttt{x : int list} \end{array} \right\} \vdash \quad \texttt{x + sum xs} \quad : \texttt{int}$$

How do we generate these declarations?

Pattern Typing Judgments

$$p : \tau \Rightarrow \Gamma$$

"The pattern p is of type τ and generates the variable declarations Γ"

Our derivations now have multiple kinds of judgments

Pattern Typing (Attempt)

Pattern Typing (From MP2)

$$\frac{}{\texttt{()} : \texttt{unit} \Rightarrow \varnothing}\ \textsc{unitP}$$

$$\frac{}{\texttt{true} : \texttt{bool} \Rightarrow \varnothing}\ \textsc{trueP}$$

$$\frac{}{\texttt{false} : \texttt{bool} \Rightarrow \varnothing}\ \textsc{falseP}$$

$$\frac{\text{n is an integer literal pattern}}{\mathsf{n} : \texttt{int} \Rightarrow \varnothing}\ \textsc{intLitP}$$

$$\frac{x \text{ is a variable pattern}}{x : \tau \Rightarrow \{x : \tau\}}\ \textsc{varP}$$

$$\frac{}{\texttt{[]} : \texttt{int list} \Rightarrow \varnothing}\ \textsc{nilP}$$

$$\frac{p_1 : \texttt{int} \Rightarrow \Gamma_1 \qquad p_2 : \texttt{int list} \Rightarrow \Gamma_2 \qquad \Gamma_1 \text{ and } \Gamma_2 \text{ are disjoint}}{p_1 \texttt{ :: } p_2 : \texttt{int list} \Rightarrow \Gamma_1, \Gamma_2}\ \textsc{consP}$$

$$\frac{p_1 : \tau_1 \Rightarrow \Gamma_1 \qquad \ldots \qquad p_k : \tau_k \Rightarrow \Gamma_k \qquad \Gamma_1, \ldots, \Gamma_k \text{ are disjoint}}{p_1 \texttt{ , } \ldots \texttt{ , } p_k : \tau_1 \texttt{ * } \ldots \texttt{ * } \tau_k \Rightarrow \Gamma_1, \ldots, \Gamma_k}\ \textsc{tupleP}$$

Pattern Typing (Example)

$$\frac{}{\texttt{()} : \texttt{unit} \Rightarrow \varnothing}\ \textsc{unitP} \qquad \frac{}{\texttt{true} : \texttt{bool} \Rightarrow \varnothing}\ \textsc{trueP} \qquad \frac{}{\texttt{false} : \texttt{bool} \Rightarrow \varnothing}\ \textsc{falseP}$$

$$\frac{\text{n is an integer literal pattern}}{\mathsf{n} : \texttt{int} \Rightarrow \varnothing}\ \textsc{intLitP} \qquad \frac{x \text{ is a variable pattern}}{x : \tau \Rightarrow \{x : \tau\}}\ \textsc{varP} \qquad \frac{}{\texttt{[]} : \texttt{int list} \Rightarrow \varnothing}\ \textsc{nilP}$$

$$\frac{p_1 : \texttt{int} \Rightarrow \Gamma_1 \qquad p_2 : \texttt{int list} \Rightarrow \Gamma_2 \qquad \Gamma_1 \text{ and } \Gamma_2 \text{ are disjoint}}{p_1 \texttt{ :: } p_2 : \texttt{int list} \Rightarrow \Gamma_1, \Gamma_2}\ \textsc{consP}$$

$$\frac{p_1 : \tau_1 \Rightarrow \Gamma_1 \qquad \ldots \qquad p_k : \tau_k \Rightarrow \Gamma_k \qquad \Gamma_1, \ldots, \Gamma_k \text{ are disjoint}}{p_1 \texttt{ , } \ldots \texttt{ , } p_k : \tau_1 \texttt{ * } \ldots \texttt{ * } \tau_k \Rightarrow \Gamma_1, \ldots, \Gamma_k}\ \textsc{tupleP}$$

Match Typing (In All)

$$\frac{}{\texttt{()} : \texttt{unit} \Rightarrow \varnothing}\ \textsc{unitP} \qquad \frac{}{\texttt{true} : \texttt{bool} \Rightarrow \varnothing}\ \textsc{trueP} \qquad \frac{}{\texttt{false} : \texttt{bool} \Rightarrow \varnothing}\ \textsc{falseP}$$

$$\frac{\text{n is an integer literal pattern}}{\mathsf{n} : \texttt{int} \Rightarrow \varnothing}\ \textsc{intLitP} \qquad \frac{x \text{ is a variable pattern}}{x : \tau \Rightarrow \{x : \tau\}}\ \textsc{varP} \qquad \frac{}{\texttt{[]} : \texttt{int list} \Rightarrow \varnothing}\ \textsc{nilP}$$

$$\frac{p_1 : \texttt{int} \Rightarrow \Gamma_1 \qquad p_2 : \texttt{int list} \Rightarrow \Gamma_2 \qquad \Gamma_1 \text{ and } \Gamma_2 \text{ are disjoint}}{p_1 \texttt{ :: } p_2 : \texttt{int list} \Rightarrow \Gamma_1, \Gamma_2}\ \textsc{consP}$$

$$\frac{p_1 : \tau_1 \Rightarrow \Gamma_1 \qquad \ldots \qquad p_k : \tau_k \Rightarrow \Gamma_k \qquad \Gamma_1, \ldots, \Gamma_k \text{ are disjoint}}{p_1 \texttt{ , } \ldots \texttt{ , } p_k : \tau_1 \texttt{ * } \ldots \texttt{ * } \tau_k \Rightarrow \Gamma_1, \ldots, \Gamma_k}\ \textsc{tupleP}$$

$$\frac{\Gamma \vdash e_0 : \tau_0 \qquad p_1 : \tau_0 \Rightarrow \Gamma_1 \qquad \Gamma, \Gamma_1 \vdash e_1 : \tau \qquad \ldots \qquad p_k : \tau_0 \Rightarrow \Gamma_k \qquad \Gamma, \Gamma_k \vdash e_k : \tau}{\Gamma \vdash \texttt{match } e \texttt{ with } \texttt{|}\ p_1 \texttt{ -> } e_1 \ldots \texttt{ |}\ p_k \texttt{ -> } e_k : \tau}\ \textsc{match}$$

Match Typing (Example)

$$\frac{\Gamma \vdash e_0 : \tau_0 \qquad p_1 : \tau_0 \Rightarrow \Gamma_1 \qquad \Gamma, \Gamma_1 \vdash e_1 : \tau \qquad \ldots \qquad p_k : \tau_0 \Rightarrow \Gamma_k \qquad \Gamma, \Gamma_k \vdash e_k : \tau}{\Gamma \vdash \texttt{match}\ e\ \texttt{with}\ \texttt{|}\ p_1\ \texttt{->}\ e_1 \ldots\ \texttt{|}\ p_k\ \texttt{->}\ e_k : \tau}\ \textsc{match}$$

Semantics

Questions

What does "matching" mean?

How do we find a match?

What do we do when we find a match?

What if we don't find a match?

Matching Semantics (Attempt)

Pattern Matching (High Level)

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

Pattern Matching (High Level)

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

Since patterns "look" like values, we just need to describe when two things "look alike"

Pattern Matching (High Level)

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

Since patterns "look" like values, we just need to describe when two things "look alike"

Reminder: matching generates variable bindings

$\{\} \vdash$

```
match 1 :: 2 :: [] with
| [] -> 0, false
| x :: xs -> x, true
```

$\Downarrow (1, \top)$

$\{\} \vdash$

```
match 1 :: 2 :: [] with
| [] -> 0, false
| x :: xs -> x, true
```

$\Downarrow (1, \top)$

find a branch that the value of the expression matches

$\{\} \vdash$

```
match 1 :: 2 :: [] with
| [] -> 0, false
| x :: xs -> x, true
```

$\Downarrow (1, \top)$

failed pattern match

$\{\} \vdash$

```
match 1 :: 2 :: [] with
| [] -> 0, false
| x :: xs -> x, true
```

$\Downarrow (1, \top)$

successful pattern match

$\{\} \vdash$

```
match 1 :: 2 :: [] with
| [] -> 0, false
| x :: xs -> x, true
```

$\Downarrow (1, \top)$

evaluate success branch

$$\{\} \vdash \texttt{x, true} \Downarrow (1, \top)$$

$\{\} \not\vdash$ `x, true` $\Downarrow (1, \top)$

We don't know what x is

$\{\} \vdash$

```
match 1 :: 2 :: [] with
| [] -> 0, false
| x :: xs -> x, true
```

$\Downarrow (1, \top)$

the pattern generates new bindings

$$\{\texttt{x} \mapsto \texttt{1}, \texttt{xs} \mapsto \texttt{[2, 3]}\} \vdash \texttt{x, true} \Downarrow (\texttt{1}, \top)$$

$$\{\texttt{x} \mapsto \texttt{1}, \texttt{xs} \mapsto \texttt{[2, 3]}\} \vdash \texttt{x, true} \Downarrow (1, \top)$$

How do we generate these declarations?

Pattern matching judgment

$$v \text{ matches } p \Rightarrow \mathscr{E}$$

"The value v matches the pattern p and generates the bindings $\mathscr{E}$"

Again, our derivations now have multiple kinds of judgments

Pattern Matching (Attempt)

Pattern Matching

$$\frac{}{\texttt{()} \text{ matches } \texttt{()} \Rightarrow \varnothing}\ \textsc{unitM}$$

$$\frac{}{\top \text{ matches } \texttt{true} \Rightarrow \varnothing}\ \textsc{trueM}$$

$$\frac{}{\bot \text{ matches } \texttt{false} \Rightarrow \varnothing}\ \textsc{falseM}$$

$$\frac{\text{n is an integer pattern for } n}{n \text{ matches n} \Rightarrow \varnothing}\ \textsc{intM}$$

$$\frac{x \text{ is an variable pattern}}{v \text{ matches } x \Rightarrow \{x \mapsto v\}}\ \textsc{varM}$$

$$\frac{}{[] \text{ matches } \texttt{[]} \Rightarrow \varnothing}\ \textsc{nilM}$$

$$\frac{n \text{ matches } p_1 \Rightarrow \mathcal{E}_1 \qquad [n_1, \ldots, n_k] \text{ matches } p_2 \Rightarrow \mathcal{E}_2}{[n, n_1, \ldots, n_k] \text{ matches } p_1 \texttt{ :: } p_2 \Rightarrow \mathcal{E}_1 \cup \mathcal{E}_2}\ \textsc{consM}$$

$$\frac{\mathcal{E} \vdash v_1 \text{ matches } p_1 \Rightarrow \mathcal{E}_1 \qquad \ldots \qquad \mathcal{E} \vdash v_k \text{ matches } p_k \Rightarrow \mathcal{E}_k}{\mathcal{E} \vdash v_1 \texttt{ , } \ldots \texttt{ , } v_k \text{ matches } p_1 \texttt{ , } \ldots \texttt{ , } p_k \Rightarrow \mathcal{E}_1 \cup \cdots \cup \mathcal{E}_k}\ \textsc{tupleM}$$

Pattern Matching (Example)

$$\frac{}{\texttt{()} \text{ matches } \texttt{()} \Rightarrow \varnothing}\ \textsc{unitM} \qquad \frac{}{\top \text{ matches } \texttt{true} \Rightarrow \varnothing}\ \textsc{trueM} \qquad \frac{}{\bot \text{ matches } \texttt{false} \Rightarrow \varnothing}\ \textsc{falseM}$$

$$\frac{\text{n is an integer pattern for } n}{n \text{ matches n} \Rightarrow \varnothing}\ \textsc{intM} \qquad \frac{x \text{ is an variable pattern}}{v \text{ matches } x \Rightarrow \{x \mapsto v\}}\ \textsc{varM} \qquad \frac{}{[] \text{ matches } \texttt{[]} \Rightarrow \varnothing}\ \textsc{nilM}$$

$$\frac{n \text{ matches } p_1 \Rightarrow \mathcal{E}_1 \qquad [n_1, \ldots, n_k] \text{ matches } p_2 \Rightarrow \mathcal{E}_2}{[n, n_1, \ldots, n_k] \text{ matches } p_1 \texttt{ :: } p_2 \Rightarrow \mathcal{E}_1 \cup \mathcal{E}_2}\ \textsc{consM}$$

$$\frac{\mathcal{E} \vdash v_1 \text{ matches } p_1 \Rightarrow \mathcal{E}_1 \qquad \ldots \qquad \mathcal{E} \vdash v_k \text{ matches } p_k \Rightarrow \mathcal{E}_k}{\mathcal{E} \vdash v_1 \texttt{ , } \ldots \texttt{ , } v_k \text{ matches } p_1 \texttt{ , } \ldots \texttt{ , } p_k \Rightarrow \mathcal{E}_1 \cup \cdots \cup \mathcal{E}_k}\ \textsc{tupleM}$$

Match Semantics (In All)

$$\frac{}{\texttt{()} \text{ matches } \texttt{()} \Rightarrow \varnothing}\ \textsc{unitM} \qquad \frac{}{\top \text{ matches } \texttt{true} \Rightarrow \varnothing}\ \textsc{trueM} \qquad \frac{}{\bot \text{ matches } \texttt{false} \Rightarrow \varnothing}\ \textsc{falseM}$$

$$\frac{\text{n is an integer pattern for } n}{n \text{ matches n} \Rightarrow \varnothing}\ \textsc{intM} \qquad \frac{x \text{ is an variable pattern}}{v \text{ matches } x \Rightarrow \{x \mapsto v\}}\ \textsc{varM} \qquad \frac{}{[] \text{ matches } \texttt{[]} \Rightarrow \varnothing}\ \textsc{nilM}$$

$$\frac{n \text{ matches } p_1 \Rightarrow \mathcal{E}_1 \qquad [n_1, \ldots, n_k] \text{ matches } p_2 \Rightarrow \mathcal{E}_2}{[n, n_1, \ldots, n_k] \text{ matches } p_1 \texttt{ :: } p_2 \Rightarrow \mathcal{E}_1 \cup \mathcal{E}_2}\ \textsc{consM}$$

$$\frac{\mathcal{E} \vdash v_1 \text{ matches } p_1 \Rightarrow \mathcal{E}_1 \qquad \ldots \qquad \mathcal{E} \vdash v_k \text{ matches } p_k \Rightarrow \mathcal{E}_k}{\mathcal{E} \vdash v_1 \texttt{ , } \ldots \texttt{ , } v_k \text{ matches } p_1 \texttt{ , } \ldots \texttt{ , } p_k \Rightarrow \mathcal{E}_1 \cup \cdots \cup \mathcal{E}_k}\ \textsc{tupleM}$$

$$\frac{\mathcal{E} \vdash e_0 \Downarrow v_0 \qquad \forall j < i (v_0 \text{ does not match } p_j) \qquad v_0 \text{ matches } p_i \Rightarrow \mathcal{E}' \qquad \mathcal{E}'' = \text{merge}(\mathcal{E}, \mathcal{E}') \qquad \mathcal{E}'' \vdash e_i \Downarrow v}{\mathcal{E} \vdash \texttt{match } e_0 \texttt{ with } \texttt{|}\ p_1 \texttt{ -> } e_1 \ldots \texttt{ |}\ p_k \texttt{ -> } e_k \Downarrow v}\ \textsc{matchE}$$

Match Semantics (Example)

$$\frac{\mathcal{E} \vdash e_0 \Downarrow v_0 \qquad \forall j < i (v_0 \text{ does not match } p_j) \qquad v_0 \text{ matches } p_i \Rightarrow \mathcal{E}' \qquad \mathcal{E}'' = \mathsf{merge}(\mathcal{E}, \mathcal{E}') \qquad \mathcal{E}'' \vdash e_i \Downarrow v}{\mathcal{E} \vdash \texttt{match}\ e_0\ \texttt{with}\ \texttt{|}\ p_1\ \texttt{->}\ e_1 \ldots\ \texttt{|}\ p_k\ \texttt{->}\ e_k \Downarrow v}\ \textsc{matchE}$$

Summary

We can formally define the syntax, typing and semantics of **match expressions**

Deep pattern matching requires the introduction of **patterns**